PAGE NO.
DATE / /

(8) Student t-Distributions.

Pghona Hai

1. What it is?

→ A normal-like distribution with thicker tails, adjusting for small sample uncertainty. (in visuals)

→ Family of curves where shape depends on degrees of freedom ($df = n-1$).

→ As $n \to \infty$, t-distribution → Normal distribution ($df > 30$+ is nearly normal)

2. Why it Matters

↳ Small samples: Corrects the normal distribution's underestimation of uncertainty.

↳ Used for:

↳ Confidence intervals for means (e.g. "Average height is 170 cm ± 5cm")

↳ t-tests (comparing two groups mean).

↳ Regression coefficient.

3. How it Works

Confidence Interval Formula (From your source).

$$\bar{x} \pm t_{n-1}(0.05) \times \frac{s}{\sqrt{n}}$$

→ $t_{n-1}(0.05)$: Critical t-value for 90% CI (5% in each tail.)

→ s: Sample standard deviation.

PAGE NO.....
DATE / /

Again (1) full Explanations:

1) The "Normal-Plus" Distribution

-> Visual: Like a normal distribution but with fatter tails (more spread at the ends)
-> Why? Small samples have extra uncertainty that normal distribution ignores.
-> Degrees of Freedom (df):
  ↳ df = n-1 (sample size minus one).
  ↳ Controls tail thickness: lower df = fatter tails.

2) Key Properties

Feature	t-Distribution	Normal distribution
Tail Thickness	Fatter (more extremes)	Thinner
Shape Control	df = n-1	Fixed shape
Use Cases	Small samples (n<30)	Large samples (n≥30)

Example
- with n=5 (df=4), a 95% CI using t is wider than normal.

Python code

3) Why Degree of Freedom Matter.

-> df = n-1 because we estimate the mean first, losing one "free" data point.

(A) The core concept:
-> Degrees of Freedom (df) = The number of independent pieces of information left after estimating

a parameter (like the mean).

- Formula: $df = n - 1$ for a sample of size n

- Why "$n-1$"?

When you calculate the sample mean ($\bar{x}$), you impose one constraint: the sum of deviations from $\bar{x}$ must be equal zero. $(x_1 - \bar{x}) + (x_2 - \bar{x}) + (x_3 - \bar{x}) = 0$.

→ Only $n-1$ data points can vary freely: the last is determined by $\bar{x}$. (Eg. Pizza price of 5 friends, where last one have no choice).

(A.1) Why "$n-1$"? Explained with an Example.

The Constraint Principle.

→ When you calculate a sample mean ($\bar{x}$), you introduce a mathematical constraints.

The sum of deviations from the mean must be equal zero.

This means:

$$\sum_{i=1}^{n} (x_i - \bar{x}) = 0.$$

→ Once you know $\bar{x}$ and $n-1$ of the data points, the n-th data point is automatically determined.

Example: Exam scores!

Let's say you have 3 exam scores and their mean is 80.

$$\bar{x} = 80$$

You freely choose the first two scores:

- Score 1: 70
- Score 2: 90.

Now, the 3rd score must adjust to ensure the mean stays at 80:

$$70 + 90 + x_3 = 80 \times 3.$$

$$x_3 = 80.$$

PAGE NO.
DATE / /

→ You had 2 "free choices" ($df = 2$) and the 3rd score was constrained by the mean.

Ans 2 Why does this affect variance & t-Distribution?

→ Sample variance (s^2) uses $\bar{x}$ (estimated from data), not the true population mean (μ)
→ Since $\bar{x}$ is calculated from the same data, it "soaks up" one degree of freedom.
→ If we used n instead of $n-1$, would underestimate true variance (Bessel's correction fixes this).

Mathematical Proof:
The sample variance formula:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})^2$$

→ Dividing by $n-1$ (not n) makes s^2 an unbiased estimator of population variance (σ^2).

Real-word Implications

1. Small samples (low df):
   - High uncertainty → t-distribution has fatter tails.
   - E.g.: with $df = 2$, 95% CI = ±4.3 SEs (vs. normal is ±1.96)

2. Large samples (high df):
   - t-distribution ≈ normal (e.g. $df = 30$, 95% CI ≈ ±2.05 SEs)

DATE / /

3 Examples

3.1 Let's try to understand Variance with real world example.

Delivery Driver & Distance

"How consistent are my drivers in their delivery routes compared to the average distance?"

Let's say

~~Driver~~		Driver	
Driver	A	B	C
Distance	8	10	12

Average (Mean) = 10 km.

Now, what's happening?

- Driver A is 2km shorter than average.
- Driver B is exactly average.
- Driver C is 2km longer than average.

These gaps from the mean are called deviations.

- A: -2, B: 0, C: +2

But if you just add those up → you get 0 (since negative cancels positives)

So instead: You square the deviations.

To get:

- A: $(-2)^2 = 4$
- B: $(0)^2 = 0$
- C: $(2)^2 = 4$

Now, you take the average of those squared deviations $\frac{4+0+4}{3} = 2.67$

This is the variance tells you on average, how far your drivers are straying (squared distance) from the mean.

Conceptually?

Variance is like saying " How spread out are these numbers from the average value?

B.2

Population Vs. Sample Variance

The whole Pizza Party of India Vs Only your city's pizza bill data

Situation 1 : The whole Pizza Party of India

→ In population dataset, the mean is known because you have access to every single value in the entire group you care about. So you you're not estimating the mean - you're literally calculating it from 100% of the data. There no guessing, no estimating. !

→ Let's come back to example of situation 1. Imagine you somehow know how much every single person paid at every pizza place across the entire country.

So all data points are truly independent. You didn't "use up" any freedom calculating the mean - it was already known.

→ So we divide by n, not n-1

$$\sigma^2 = \sum_{i=1}^{n} \frac{(x_i - \bar{x})^2}{N}$$

Situation 2 : Only your city's pizza bill data

→ Now you're trying to estimate the variance of the entire country's pizza spending using just

5 people from your city.

You don't know the true mean (μ), so you:

1) calculate the sample mean ($\bar{x}$) from your 5 friends
2) Then compute the deviation from this estimated mean.

But here's the catch:

→ The sample mean is itself calculated from the data, so it's not an independent value. It consumes one degree of freedom.

Let's get more deep down with this concepts:

Situation 2. (A) Only 4 out of 5 paid the bill.
(B) All 5 out of 5 paid the bill.

(A) Only 4 out of 5 paid the bill

Friends	A	B	C	D	E
payment	$8	$9	$11	$12	?

Five friends go out for pizza and decide to split the bill evenly. Total bill is $50, so each person must pay $10 on average.

Total must be $50:

So, E must pay: $50 - (8 + 9 + 11 + 12) = 50 - 40 = 10$.

You didn't get to choose what E paid, it was automatically fixed by

- The total bill
- And the payment of the other 4 friends.

PAGE NO.....
DATE / /

Connecting to statistics

→ Your "data points" are the 5 payment.
→ Your "mean" is the average amount: $10.
→ Because the total is fixed at $50, and you calculated the mean from the data, only 4 out of 5 payments can be independently chosen.
→ The 5th value has no freedom - it's locked by the condition that the mean is $10.

This is why in statistics we say:
"You lost 1 degree of freedom when you estimated the mean from the sample".

B.) All 5 out of 5 paid the bill

Friends	A	B	C	D	E
payment	$8	$9	$11	$12	$10.

Let's say:
↳ Five friends go out for pizza.
↳ Each one pays something different.
↳ You collect all 5 amount paid and find the average from these.

→ Even if all 5 friends paid, if the mean come from their payments, one of them is not truly "free" - they are bounded by the constraint of the mean. That is why you still lose 1 degree of freedom and divide by n-1.

Conclusion of situation 1 and situation 2

so we divid by 1

$$s^2 = \frac{\sum_{i=1}^{n} (x_i - \bar{x})^2}{n-1}$$

Finally from situation 1 and situation 2, we conclude

-) we use 'n' in population variance because the mean is fixed and known, so all values are free to vary. we use $(n-1)$ in sample variance because the mean is estimated from the sample, so 1 degree of freedom is used up in that estimation.

Similarly DOF of following are:

	parameters Estimated	DOF
Sample variance	mean (1) (1 friend fixed)	$(n-1)$.
Linear regression	Slope + intercept (2). (2 friend fixed)	$(n-2)$.
ANOVA with k group	k group means (k)	$(n-K)$.

(4. Confidence Intervals with t-Distribution: Formulas & worked Example).

(1) Mathematical Formulas.

For a $(1-\alpha)$% confidence interval around the same mean $\bar{x}$ (when population variance is unknown).

$$CI = \bar{x} \pm t_{\alpha/2, df} \cdot \frac{s}{\sqrt{n}}$$

$s/\sqrt{n}$ = Standard Error

$\bar{x}$ = Sample mean.

s = Sample standard deviation.

n = Sample size.

df = $n-1$: Degrees of freedom.

$t_{\alpha/2, df}$ = Critical t-value (cuts off $\alpha/2$ in each tails).

2. Worked Example

Problem

Calculate a 90% confidence interval for the following sample of 5 measurements.

Step 1

Data = [12.1, 13.4, 11.7, 12.8, 13.0]

1. Sample mean ($\bar{x}$) = 12.6.
2. Sample std dev(s) = 0.689.
3. Std Error (SE) = $\frac{s}{\sqrt{n}} = \frac{0.689}{\sqrt{5}} = 0.308$

Step 2. Find Critical t-value

(a) Clarifying the Confidence level and α (Alpha),

CI vs α

CI - The probability that the confidence interval contains the true population parameter (e.g. 90%, 95%)

α - The probability that the interval does not contain the true parameter (i.e. the error rate).

Relationship

$$\alpha = 1 - CI$$

- For 90% Confidence

$$\alpha = 1 - 0.90 = 0.10.$$

- For 95% Confidence

$$\alpha = 1 - 0.95 = 0.05.$$

(b) Two-Tailed vs. One-Tailed Test.

- Two-tailed test: used when you care about deviations in both directions (e.g. "Is the mean different from x")
  - ↳ The error rate α is split equally between both tails.
  - ↳ For $\alpha = 0.10$:

Each tail = $\alpha/2 = 0.05$.

- One-tailed test : Used when you only care about one direction. (e.g. "Is the mean greater than X?").

↳ The full 'α' is placed in one tail.

② Finding the critical t-value

For a 90% two-tailed confidence interval.

1. Total $\alpha = 0.10$ (10% error rate).
2. Split $\alpha/2 = 0.05$ into each tail.
3. Cumulative probability
   ↳ The t-value should leave 95% of the distribution to its left (since 5% in the upper tail)

python code

```
from scipy.stats import t

df = 4 # Degrees of freedom
t_crit = t.ppf(0.95, df) # 0.95 = 1 - α/2
print(t_crit) # output: 2.132
```

Interpretation

- A t-value of 2.132 means:
  - 5% of the distribution lies above +2.132.
  - 5% lies below -2.132.
  - 90% lies between -2.32 and +2.132.

Step ③ Calculate CI

$CI = 12.6 \pm 2.132 \times 0.308$

Margin of Error $= 0.657$

$CI = [12.6 - 0.657, 12.6 + 0.657] = [11.94, 13.26]$

Thus "we are 90% confident the true population mean lies between 11.94 and 13.26"

PAGE NO.....
DATE / /

5. Gosset's Resampling Experiment

- Method: Drew 3,000 random samples (size n = 4) from criminal centropometry data.
- Finding: Standardized sample means followed the t-distribution, not the normal.
- Legacy: t-distribution became the gold standard for small-sample inference.

6. Gosset Modern Relevance for Data Scientists

- Classical stats: t-test, regression, A/B testing rely on t-distribution.
- Data Science: While bootstrapping is often preferred, t-distribution still appear in:
  - ↳ Statistical software output (e.g. p-value in regression.
  - ↳ Experiment design (e.g. calculating sample size).

PAGE NO.
DATE / /

Brief: Empirical rule vs. Solo CI vs CI with z-test vs CI wit t-test.

Real-World Analogy (Pizza Delivery Drivers):

Let's say you collect data on how long it takes 25 drivers to deliver pizzas.

- Sample Mean = 40 minutes.
- Sample std Dev = 7.07 minutes.
- Sample Size = 25.

(1) Empirical Rule (68-95-99.7 Rule).

This helps you understand how the data is distributed:

- Mean ± 1 std → 68% drivers take [32.93, 47.07] min.
- Mean ± 2 std → 95% drivers take [25.86, 54.14] min
- Mean ± 3 std → 99.7% drivers take [18.79, 61.21] min

[✓] Focus: Spread of individual data points assuming normality.

[✗] Doesn't estimate the true mean of the population.

(2) Solo Confidence Interval (CI without test).

You want to estimate where the true average delivery time lies based on sample data.

- SE = std/√n = 7.07/√25 = 1.414.
- z-value (for 95%) = 1.95
- CI = 40 ± 1.96 × 1.414 = [37.23, 42.77]

[✓] We are 95% confident the true population mean lies between 37.23 and 42.77 minutes.

[✗] No hypothesis test involved, just estimation.

PAGE NO.....
DATE / /

(3) CI with Z-test.

purpose: CI + Hypothesis Testing, Large sample size ($n \geq 30$), known pop std.

Applies when: Normal distribution, large sample, known population σ

→ You're doing the same CI, but also want to test if your sample support a claim: "Average delivery time is 45 min". You use a Z-test with the same data:

- Null Hypothesis (H_0): $\mu = 45$.
- Alternative (H_1): $\mu \neq 45$.

$$Z = (\bar{x} - \mu) / SE = (40 - 45) / 1.414 = -3.53.$$

Since $Z = -3.53$ is beyond ± 1.96, you reject H_0.

✓ And the CI (37.23 to 42.77) doesn't contain 45, so it agrees with the test.

(4) CI with T-test

purpose: CI + Hypothesis Testing, small sample size ($n < 30$), unknown pop std.

Applies when: Small sample, or unknown σ

→ Suppose you only had $n = 9$ drivers, and σ (population std) is unknown

- $\bar{x} = 40$
- $s = 7.07$
- $SE = 7.07 / \sqrt{9} = 2.36$
- DOF $= 9 - 1 = 8$
- $t(0.05, 8) = 2.306$.

DATE / /

Python code 1

```
From scipy import stats
Import numpy as np

# Sample data (heights in cm).
sample = np.array([168, 170, 172, 169, 171])
    n = len(sample).
mean, std = np.mean(sample), np.std(sample, ddof=1)
    # ddof=1 for sample std.

# 90 % CI using t- distribution.
    Confidence_level = 0.90
    alpha = 1 - confidence_level.
    ~~t_crit = stats.t.ppf~~
    t_crit = stats.t.ppf(1 - alpha/2, df=n-1)
    # critical t- value for 90% CI and n-1 degrees
        of freedom.

# Margin of error
    margin_of_error = t_crit * (std/np.sqrt(n)).

# Confidence interval.
    ci = (mean - margin_of_error, mean + margin_of_error)

print(f"90% confidence interval: [{ci[0]:.1f}, {ci[1]:.1f}]")
```

what's happening!

- confidence level = 0.90 → means you want 90% of the probability in the centre.
- alpha = 1 - 0.90 = 0.10 → leftover 10% error is split equally across both tails → 5% in the left tail, 5% in the right tail.

PAGE NO.....
DATE / /

- alpha/2 = 0.05 → upper tail cutoff is at 95% cumulative probability.
- stats.t.ppf(1 - alpha/2, df=n-1) → finds the t-value that cuts off the top 5% tail.

But we want CI of 90% - why is 1 - alpha/2

Let's think in tiny steps:

Suppose you want 90% area in the middle of the distribution what's left over?

→ 100% - 90% = 10%

→ This leftover 10% is split equally into two tails.

→ So, 5% is in the left tail and 5% is in the right tail.

Like this

|—— 5% ---|--- 90% ---|--- 5% ---|

Now -- when you ask for the critical t-value, you need to find:

→ The point when left side cumulative probability = 95%

Because:

→ Everything left of that point is 5% (left tail) + 90% (middle) = 95%

That's why 0.95 ✓

The critical value at cumulative probability 95% will "chop off" the top 5%

DATE / /

Math in steps

- Confidence level = 90% ⇒ center part = 90%
- Alpha = 1 − 0.90 = 0.10 ⇒ leftover 10%
- Each tail = 0.10/2 = 0.05 = 5% in each tail.
- Cumulative probability at right edge of center = 1 − 0.05 = 0.95.

Thus:

t_crit = stats.t.ppf(1 − alpha/2, df = n−1)

becomes

t_crit = stats.t.ppf(0.95, df = n−1).

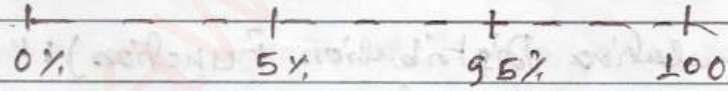
because 1 − (0.10/2) = 0.95.

Short summary again:

You want 90% in the center.

leftover 10% is split into two tails: 5% + 5%

The cutoff on the right is at cumulative probability = 95%



- Left 5%: Tail.
- Middle 90%: Your CI.
- Right 5%: Tail.
- Critical value: 95% cumulative,

PAGE NO.
DATE / /

Another 80% CI

If you want to find 80% CI then $1 - 0.20/2$, $df = n-1$

- correct.
- 80% CI ⇒ alpha = $1 - 0.80 = 0.20$.
- Each tail = $0.20 / 2 = 0.10$

Now:

$1 - 0.10 \rightarrow 0.90 \rightarrow ppf(0.90, df = n-1)$.

✓ Yes, so you call:

t-crit = stats.t.ppf(0.90, df = n-1).

You continued:

that means ppf is cumulative, it adds 0.10 and 0.80 → total 0.90 will be found.

Exactly

- ppf stands for Percent Point Function – it is the inverse of CDF
- CDF (Cumulative Distribution Function): "How much area is to the left of this t-value?"
- So when you do **ppf(0.90, df)**, you are asking
  → Find the t-value where the total area under the curve to the left is 90%

✓ which includes

- 10% (left tail)
- 80% (middle).

(10% + 80% = 90%)

DATE / /

confidence Interval	Alpha	Tail Area Each	ppf() Each.
90%	0.10	0.05	0.95.
80%	0.20	0.10	0.90.
95%	0.05	0.025	0.975.
99%	0.01	0.005	0.995.

In simply

PPF(x) = "Tell me the t-value where total left area = x".

Different ways to find t-critical

(1) Using scipy.stats.t.ppf().

```
t_crit_two_tailed = stats.t.ppf(1 - alpha/2, df=df)
print(t_crit_two_tailed)
```

(2) Using statsmodels.stats.weightstats._tconfint_generic()

```
import statsmodels.stats.weightstats as smstats
t_crit = smstats._tconfint_generic(0, 1, alpha,
        df, alternative='two-sided')[1]
print(t_crit)
```

(3) using scipy.stats.t.isf() (Inverse Survival function).

```
t_crit = stat.t.isf(alpha, df=df).
print(t_crit)
```

(2) Why we use $\sqrt{n}$ and not $\sqrt{n-1}$ in the margin of error formula.

PAGE NO.
DATE / /

margin_of_error = t_crit * (std / np.sqrt(n))

Why $\sqrt{n}$ is Correct (and $\sqrt{n-1}$ is Wrong here)?

(1) [std] already accounts for n-1 (Bessel's correction)

→ When you compute [std] with ddof = 1 (default in many libraries), it uses:

$$s = \sqrt{\frac{\sum (x_i - \bar{x})^2}{n-1}}$$

→ This corrects bias in small samples, making [std] an unbiased estimator of the population standard deviation (σ).

(2) Standard Error (SE) uses $\sqrt{n}$ (not $\sqrt{n-1}$).

→ The standard error measures the variability of the sample mean, not individual data points.

→ The $\sqrt{n}$ comes from the central limit Theorem (CLT) which says:

↳ The standard deviation of the sampling distribution of the mean is $\sigma/\sqrt{n}$ (estimated by $s/\sqrt{n}$)

↳ Larger n → tighter clustering of sample means around the true mean (μ).

(3) $\sqrt{n-1}$ would overcorrect and give wrong uncertainty.

⇒ Using $\sqrt{n-1}$ would artificially inflate the margin of error, making confidence intervals too wide

⇒ Example: If n=5, $\sqrt{5} \approx 2.24$ vs $\sqrt{4} = 2$. Using $\sqrt{4}$ would make the margin of error ~ 12% larger than it

PAGE NO.....
DATE / /

Plot t-distribution vs. normal distribution.

x = np.linspace(-4, 4, 1000).
t_pdf = stats.t.pdf
Import numpy as np.
import scipy.stats as stats.
import matplotlib.pyplot as plt

Generate 1000 points between -4 and 4 (std dev from the mean)
This is where you visualize the distributions over a smooth, continuous range
1000 points from -4 to 4
x = np.linspace(-4, 4, 1000). (1)

Calculate the PDF for t-distribution with df.
t-distribution with 3 degrees of freedom
t_pdf = stats.t.pdf(x, df=3) (2)

~~# Calculate the pdf for t-distribution with df=3~~
calculate the PDF for the standard normal distribution (df=∞, mean=0, std=1)
normal_pdf = stats.norm.pdf(x) #standard (3)

Plot the distributions
plt.figure(figsize=(10,6)).

Plot the t-distributions
plt.plot(x, t_pdf, label='t-distribution (df=3)', color='blue', linewidth=2)

Plot the standard normal distribution
plt.plot(x, normal_pdf, label='Standard Normal', color='red', linestyle='--')

PAGE NO.....
DATE / /

Title and labels.
plt.title('t-distribution vs. Normal distribution', fontsize=14)
plt.xlabel('Standard deviations from Mean', fontsize=12).

Add a legend.
plt.legend(fontsize=12).

Display grid
plt.grid(True, alpha=0.3).

Show the plot.
plt.show().

dhanya dorle ghising

1. What does x = np.linspace(-4, 4, 1000) do?

→ x = np.linspace(-4, 4, 1000) generate 1000 evenly spaced points between -4 and 4. This range represents standard deviations from the mean, covering the majority of both the Normal and t-distribution.

→ Why 1000 points? More points make the plot smoother, allowing you to better visualize the distribution.

(2) How does df = 3 work in stats.t.pdf(x, df = 3)?

→ In the stats.t.pdf(x, df = 3), df = 3 indicates that we are working with a t-distribution where the degree of freedom are 3.

→ Degree of freedom df are calculated as n-1, where n is the sample size. Here, we're assuming n = 4, so df = 4 - 1 = 3.

→ The function calculate the probability density (y-value) for each x-point in the array based on the t-distribution formula.

↳ What does stat.t.pdf() do?

→ stats.t.pdf(x, df = 3) calculates the probability density at each x-value for a t-distribution with 3 df. This result is a y-value representing the likelihood of the

x-values occurring in the t-distribution.

↳ why is df important for the t-distribution.
⇒ The df affect the shape of the t-distribution
. As the degree of freedom increases, the t-distribution approaches the normal distribution
. For small sample size (like n=4), the t-distributions is wider and more spread out, which accounts for the uncertainty in small samples.

↳ What's the difference between the t-distribution and the normal distributions?
→ The t-distribution is used when you have small sample size (usually n < 30) and you don't know the population standard dev.
It has fatter tails to accounts for the additional uncertainty.
→ The normal distribution assumes a large sample size or known population parameters. It's narrower compared to the t-distribution for small sample sizes.

(8) what does stats.norm.pdf(x) do?
→ stats.norm.pdf(x) calculates the probability density for the standard normal distribution at each x-value. The normal distribution has mean = 0 and Standard dev = 1.

→ No [df] in stats. norm. pdf(x) because the normal distribution does not have degrees of freedom. It is fully specified by the mean and standard deviation.

→ [df] is used for only in t-distribution to account for small sizes and uncertainty.

Conclusion

Why are the plots of t-distribution and normal distribution different?

→ The t-distributions has wider than the normal distributions, especially for small degrees of freedoms (e.g. df ≥ 3). This accounts for the uncertainty in estimating the population parameters from a small sample.

→ As the degrees of freedoms increase (e.g. df ≥ 30), the t-distributions looks more like the normal distribution.

~~what is~~

Types of t-test

1) One-sample t-test.
2) Independent two-sample t-test.
3) Paired sample t-test.

Assumption of the t-test

- The data should be approximately normally distributed.
- The data should be independent (for the independent two-sample t-test).
- Homogeneity of variance (for independent two-sample t-test) - the variance of the two groups should be roughly equal.

How to perform a t-test (steps):

1) State the hypotheses: H_0 and H_A
2) Choose significance level (α): Commonly, 0.05 is used.
3) Calculate the test statistic: Use the appropriate t-test formula.
4) Find the degrees of freedom: Based on the sample size (s)
5) Find the critical value or p-value: Compare the t-statistic to the critical value (from t-distribution tables) or compute the p-value
6) Make a decision: If the p-value is less than α, reject the null hypothesis. If the t-statistic exceeds the critical value, reject the null hypothesis.

(1) One-sample t-test?

A one-sample t-test is a statistical method used to determine whether the mean of a single sample is significantly different from a known or hypothesized population mean (μ_0).

Used when:

→ You have one sample of numeric data.
→ The population standard deviation is unknown
→ The population is approximately normally distributed (especially important when the sample size is small).

Formula

The t-score is calculated as:

$$t = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}$$

df: $df = n - 1$

Example

Suppose
→ sample: [38, 36, 40, 37, 39].
→ μ_0 (hypothesized mean) = 38.5.

Step 1, Sample stats.

→ $\bar{x} = \frac{38+36+40+37+39}{5} = 38.0$.

→ $s \approx 1.58$

→ $n = 5$.

Step2

Compute t-score.

$$t = \frac{38.0 - 38.5}{1.58/\sqrt{5}} \Rightarrow \frac{-0.5}{0.707} \approx 0.71.$$

Step3

Check critical value ($df = 4, \alpha = 0.05 \rightarrow$ two-tailed $\Rightarrow \pm 2.776$)

Conclusion:

Since -0.71 is within -2.776 and $+2.776$, we fail to reject the null hypothesis.

Python code : Combined code:

```
import numpy as np
import math
from scipy import stats.

# Data
Sample = np.array([38, 36, 40, 37, 39])
mu_0 = 38.5.
  n = len(sample).
# Manual
# Sample mean:
Sample_mean = np.mean(sample)

# Sample standard deviation (with Bessel's
    corrections)
sample_std = np.std(sample, ddof=1).
```

```
# t-statistic formula:
t-stat = (sample_mean - mu_0) /
         (sample_std / math.sqrt(n))

# Degree of freedom
df = n - 1

# Two-tailed p-value from t-distribution.
t-stat = (sample_mean - mu_0) / (sample_std / math.
         sqrt(n)).
#Two-tailed p-value from t-distribution.
p-value = 2 * stats.t.sf(np.abs(t-stats), df)

# Using stats.ttest
t-stat, p-value = stats.ttest_1samp(.
                  sample, popmean = mu_0).

# Print results
print(f"Sample Mean : {sample_mean:.2f}")
print(f"Sample std dev : {sample_std:.2f}"
print(f"Sample Size: {n}").
print(f"T-statistic : {t-stat:.3f}")
print(f"Degree of freedom : {df}")
print(f"P-value : {p-value:.4f}").
```

output

Sample Mean = 38.0

Sample std dev = 1.58

T-Stat = -0.707

p-value = 0.515 (non-significant at $\alpha = 0.05$)

~~Summary Table~~

Test type and their code for p-value

Test type	Hypotheses	Code for p-value
Two-tailed	$H_0: \mu = \mu_0$ $H_1: \mu \neq \mu_0$	2 * stats.t.sf(abs(t-stat), df).
One-tailed (>)	$H_0: \mu \leq \mu_0$ $H_1: \mu > \mu_0$.	stats.t.sf(t-stat, df)
one-tailed (<)	$H_0: \mu \geq \mu_0$. $H_1: \mu < \mu_0$	stats.t.cdf(t-stat, df).

dhawa done digital

2. Independent Two-Sample t-test.

Concepts

An independent two-samples t-test is used to determine whether the mean of two independent groups are significantly different.

~~This test assumes~~

A independent two-sample t-test is used to determine whether the means of two independent groups are significantly different.

This test assumes:

- Independence: Groups have different subjects.
- Normality: Each group's data is approximately normally distributed.
- Equal variances (for standard version).

Hypothesis (where df = len(group1) + len(group2) - 2

(A) Two-tailed Test.

- $H_0: \mu_1 = \mu_2$.
- $H_1: \mu_1 \neq \mu_2$.

> 2 * stats.t.sf(abs(t-stat), df).

(B) One-tailed Test (Right-Tailed).

- $H_0: \mu_1 \leq \mu_2$.
- $H_1: \mu_1 > \mu_2$.

> stats.t.sf(t-stat, df)

(C) " " " (Left-Tailed).

- $H_0: \mu_1 \geq \mu_2$
- $H_1: \mu_1 < \mu_2$

> stats.t.cdf(t-stat, df)

Formula

t-score (equal variances assumed).

$$t = \frac{\bar{X}_1 - \bar{X}_2}{s_p \cdot \sqrt{\frac{1}{n_1} + \frac{1}{n_2}}}$$

where:

- $\bar{X}_1, \bar{X}_2$: sample means.
- n_1, n_2 : sample sizes
- s_p : Pooled standard deviation

$$s_p = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n_1 + n_2 - 2}}$$

Degree of freedom:

$$df = n_1 + n_2 - 2$$

if variance are unequal, welch's t-test is used instead (Python default to this)

Feature	Student t-test	Welch's t-test
Assumes equal variances	Yes	No
Equal sample sizes needed?	Preferable but not required	Not necessary

More robust when	Variances are equal	Variances are unequal or unknown.
In python (scipy)	equal_var=True	equal_var=False (default=False)

Syntax

```
from scipy import stats.
t-stat, p-value = stats.ttest_ind(group1,
                  group2, equal_var=False).
```

Testing whether variances are equal before choosing which test.

To determine whether the two groups have equal variances before choosing between student's t-test and Welch's t-test, we commonly use Levene's Test or F-test.

(1). Levene's Test (Robust to non-normality).

This is the preferred method.

```
from scipy.stats import levene

stat, p = levene(group1, group2).

if p > 0.05:
    print("Fail to reject Ho: Variance
          are equal --> use student's t-test")
```

else:

print ("Reject Ho: Variances are not equal
→ use welch's test")

① ~~Two-Tai~~ Two-Tail

Finding hypothesis according to p-value

From scipy import stats.

t-stat, p = stats.ttest_ind(group1,
group2, equal_var = True)

equal_var = True → equal variance → student's t-test.

use

equal_var = False → unequal variance → welch's t-test.

① Two-Tailed Test: $H_1 \neq H_2$

p_value = p. # already gives two-tailed p-value

② One-Tailed Test (Right-tailed) $H_1 > H_2$

H_1: group1 > group2

if t-stat > 0:

p_value = p/2

else:

p-value = 1 - (p/2)

(3) One-Tailed Test (left-tailed)

$\mu_1 < \mu_2$

H_1: group1 < group2

if t_stat < 0:

p-value = p/2

else:

p-value = 1 - (p/2)

Python Manually

```
import numpy as np
from scipy import stats

group1 = np.array([55, 60, 65, 62, 58, 59,
                   63, 61, 64, 57])
group2 = np.array([52, 56, 53, 55, 54, 50, 52,
                   51])

mean1, mean2 = np.mean(group1), np.mean
               (group2)
std1, std2 = np.std(group1, ddof=1), np.
             std(group2, ddof=1)

n1, n2 = len(group1), len(group2)

# Pooled stand
sp = np.sqrt((n1-1)*std1**2 + (n2-1)
     *std2**2/(n1+n2-2))
```

t-statistic

t-stat = (mean1 - mean2) / sp * np.sqrt(1/n1 + 1/n2))

Degree of freedom.

df = n1 + n2 - 2.

Two-tailed p-value

p-value = 2 * stats.t.sf(np.abs(t-stat), df)

print(f"t-statistics: {t-stat:.4f}")
print(f"Degree of freedom: {df}").
print(f"p-value: {p-value:.4f}")

output,

t-stat: 5.7622

df = 16

p-value: 0.000.

(3) Paired Sample t-Test

Pairs'

→ The Paired Sample t-Test (also called Dependent t-Test) compares the means of two related groups. This test is used when the same subjects are measured twice - under two conditions, or at two different times.

Use Cases:

- Before vs. After treatment.
- Measurements under two conditions on the same persons.
- Matched samples (e.g. twins, matched pairs in experiments)

Hypotheses

let $d = x_{after} - x_{before}$

- Null Hypothesis (H_0): $\mu_d = 0$ → No change in Mean.
- Alternative Hypothesis (H_1):
  - Two-tailed: $\mu_d \neq 0$
  - Left-tailed: $\mu_d < 0$ (e.g. decrease after treatment)
  - Right-tailed: $\mu_d > 0$ (e.g. improvement after treatment).

Formulas

(1) Differences

$d_i = x_i \text{ after} - x_i \text{ before}$.

(2) Mean of difference

$$\bar{d} = \frac{1}{n} \sum d_i$$

(3) Standard deviation of differences

$$s_d = \sqrt{\frac{\sum (d_i - \bar{d})^2}{n-1}}$$

(4) t - statistic

$$t = \frac{\bar{d}}{s_d / \sqrt{n}}$$

(5) $df = n - 1$.

Python (A) Manual

```
import numpy as np.
from scipy import stats.
import matplotlib.pyplot as plt.

# Example Data (Before & After Treatment).
before = np.array([120, 115, 130, 125, 128])
after = np.array([118, 112, 121, 125])

# Step 1: Difference
diff = after - before.
```

#step2 : Mean & std of Difference

~~d_bar = np.mean & std of differences.~~

d_bar = np.mean(diff).

s_d = np.std(diff, ddof=1).

n = len(diff)

step4: Degree of freedom and p-value (two-tailed).

df = n-1

p_value = 2 * stats.t.sf(np.abs(t_stat), df)

print(f"Mean of differences: {d_bar:.4f}")
print(f"standard deviation: {s_d:.4f}")
print(f"t-statistic: {t_stat:.4f}").
print(f"Degree of freedom: {df}")
print(f"p-value: {p_value:.4f}").

Using Built-in Function.

t_stat, p_val = stats.ttest_rel(after, before)
print(f"t-stat: {t_stat:.4f}, p-value: {p_val:.4f}")

output

mean diff: -3.000

std : 0.7071

t-statistic: -9.4868

df : 4

p-value = 0.0007

For manually

Test Type	Alternative Hypothesis	Tail Direction	Code for p-value
Two-tailed	$\mu_d \neq 0$	Change in either direction	2 * stats.t.sf(abs(t-stat), df)
One-tailed (greater)	$\mu_d > 0$	After > Before	stats.t.sf(t-stat, df).
One-tailed (less)	$\mu_d < 0$	After < Before	stats.t.cdf(t-stat, df).

For Built-in Functions

t-stat, p-val-two-tailed = stats.ttest_rel(after, before).

df = len(after) - 1.

For two-tailed:

Already done by default.

print("Two-tailed p-value:", p-val two-tailed).

For one-tailed (after > before):

use .sf() (survival function / 1 - CDF):

p-val-greater = stats.t.sf(t-stat, df).

For one-tailed (after < before):

use .cdf()

p-val-less = stats.t.cdf(t-stat, df).